

Simulation project

Mitas Stavros, Ashe Raymond-Barker

March 2026

Executive summary

This report evaluates the operational performance of BoxCar’s ride-sharing app in Squareshire and proposes data-driven improvements to increase rider satisfaction and driver productivity. A discrete-event, object-oriented, simulation model was developed to replicate the driver-rider matching process used by the BoxCar app, including spatial trip generation, driver availability dynamics, rider patience behaviour, and travel time variability.

First, a baseline system was implemented using the assumptions provided by BoxCar. Empirical rider and driver datasets from December 2025 and early January 2026 were then analysed to test these assumptions and estimate more realistic model parameters. The analysis revealed serious discrepancies between assumed and observed behaviour. The biggest disparities were found in the distribution of rider origins and destinations, and driver initial locations. Analysis found that these were better modelled by beta distributions than uniform distributions. Other differences identified in the parameters for driver and rider inter-arrival rates, driver online-time distributions and trip characteristics. Incorporating these empirically derived parameters produced a calibrated simulation that more accurately reproduces observed system outcomes.

The calibrated model was used to evaluate system performance using key operational metrics, including rider abandonment rates, rider waiting times, driver hourly earnings, and fairness across drivers. Simulation experiments demonstrate that the current system design leads to inefficiencies in driver utilisation and avoidable rider abandonments.

Based on these findings, the report proposes several modifications to the platform’s operational logic. These include extra fees for excessively long distance journeys and improved rider–driver matching that takes into account customer waiting times on top of distance and the introduction of matching riders to a busy driver who is dropping off their customer soon and nearby. Simulation results indicate that these changes can improve overall system efficiency while increasing driver hourly average earnings by 13.6%, reducing standard deviation of these hourly earnings by 11.1% and reducing rider abandonment by 29.9%.

1 Basic Model

We first constructed a basic simulation model directly from the operating assumptions provided by BoxCar. The purpose of this model is to represent how the system would perform if the company’s stated assumptions about driver behaviour, rider demand, and trip characteristics were taken at face value. It therefore serves as a baseline against which later refinements and data-based calibrations can be compared.

The platform is represented as a discrete-event simulation. In this framework, time advances from one event to the next rather than continuously. Events correspond to moments at which the system changes, such as driver arrivals, rider requests, rider abandonment, pickup completion, and trip completion. Whenever an event occurs, the simulation clock moves to that time and the state of the system is updated.

The system operates in a square region representing Squareshire. Locations within the region are described by two-dimensional coordinates $x = (x_1, x_2)$. Distances between locations are measured using Euclidean distance:

$$d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}.$$

Driver arrivals are assumed to follow a Poisson process with rate $\lambda_d = 3$ drivers per hour. When a driver enters the system, an initial location L_i is sampled uniformly across the region, and the driver remains online for a random duration

$$H_i \sim U(5, 8) \text{ hours.}$$

Rider requests also arrive randomly with rate $\lambda_r = 30$ requests per hour. Each request j generates a uniformly sampled pickup location O_j , a dropoff location D_j , and a patience time

$$P_j \sim \text{Exp}(\lambda_p),$$

where $\lambda_p = 5$ per hour. If a rider is not matched with a driver before the deadline $t_{\text{request},j} + P_j$, the request leaves the system and is counted as an abandonment.

When a rider request occurs, the platform attempts to assign the nearest available driver. Let $A(t)$ denote the set of available drivers at time t . The dispatch rule selects

$$i^* = \arg \min_{i \in A(t)} d(L_i, O_j),$$

where L_i is the location of driver i and O_j is the pickup location of rider j . If no driver is available, the rider joins the waiting queue until either a driver becomes available or the patience deadline is reached.

Once a rider and driver are matched, the journey takes place in two stages. First, the driver travels from the current location to the pickup point. Second, the rider is transported from the pickup point to the destination. Travel time depends on distance and an assumed average speed of $v = 20$ miles per hour. To allow for variation in traffic and routing conditions, the realised travel time is sampled uniformly around the expected travel time:

$$T \sim U(0.8\mu_t, 1.2\mu_t),$$

where μ_t denotes the expected travel time implied by distance and speed.

The model also includes a simple economic structure for each trip. Let d_{pickup} denote the distance from the driver to the pickup location and d_{service} denote the distance from pickup to dropoff. The passenger fare is defined as

$$\text{Fare} = 3 + 2d_{\text{service}},$$

while the driver's fuel cost is

$$\text{Cost} = 0.2(d_{\text{pickup}} + d_{\text{service}}).$$

The driver's net earnings from the trip are therefore

$$\text{Net} = \text{Fare} - \text{Cost}.$$

The simulation evolves by repeatedly processing events in chronological order. At each step, the earliest scheduled event is removed from the event list, the simulation clock advances to that time, and the system state is updated. Processing an event may also generate new future events, such as pickup completion, trip completion, or rider abandonment.

During each simulation run, the model records rider waiting times, completed trips, abandoned requests, and driver earnings. These are used to compute performance measures such as the average waiting time, abandonment rate, average driver earnings, and earnings per hour. Together, these metrics provide a baseline assessment of the BoxCar system under the company's initial assumptions.

Pseudocode Representation of the Basics Model

This section presents the pseudocode for the simulation model described above. Its purpose is to show the logical structure of the simulator in a clear and implementation-independent form. Rather than repeating the full model assumptions, the pseudocode focuses on how the simulation operates: how events are scheduled, how drivers and riders are matched, and how trips are created within the event-driven framework.

The pseudocode is organised into two main parts. First, a set of shared helper procedures is defined. These procedures capture repeated operations such as event insertion, arrival scheduling, nearest-neighbour matching, trip construction, and assignment. Second, the main event-driven simulation algorithm is presented.

For notation, let A denote the set of available drivers and W the set of waiting riders. Let $\mathcal{E}$ denote the event list ordered by event time, and let $\mathcal{D}$ and $\mathcal{R}$ denote the collections of driver states and rider states, respectively. Furthermore, let L_i denote the current location of driver i , while O_j and D_j denote the pickup and dropoff locations of rider j . The values T_j^{req} , T_j^{pickup} , and T_j^{drop} represent the request, pickup, and dropoff times of rider j .

Shared Helper Procedures

The following procedures are used repeatedly within the simulation. Defining them separately makes the main event-driven algorithm more compact and easier to follow.

Event insertion. Because the simulation is event-driven, the event list is a central data structure. Each event is inserted with a time stamp, an event type, and an optional payload. A sequence counter is included to ensure consistent ordering when multiple events occur at the same time.

Algorithm 1 Insert Event into Priority Queue

- 1: **Input:** Event time t , event type typ , payload info, event list $\mathcal{E}$
 - 2: **Initialize:** Sequence counter $\text{seq} \leftarrow \text{seq} + 1$
 - 3: Insert $(t, \text{prio}(\text{typ}), \text{seq}, \text{typ}, \text{info})$ into $\mathcal{E}$
 - 4: **Output:** Updated event list $\mathcal{E}$
-

Arrival scheduling. Driver and rider arrivals are generated stochastically over the simulation horizon. The next arrival of each type is sampled from an exponential interarrival distribution.

Algorithm 2 Schedule Next Arrival

- 1: **Input:** Current time T_{NOW} , arrival type typ , horizon end time T_{END}
 - 2: **if** $\text{typ} = \text{driver_arrival}$ **then**
 - 3: Generate interarrival time $\Delta \sim \text{Exp}(\lambda_d)$
 - 4: **else**
 - 5: Generate interarrival time $\Delta \sim \text{Exp}(\lambda_r)$
 - 6: **end if**
 - 7: $T_{\text{NEXT}} \leftarrow T_{\text{NOW}} + \Delta$
 - 8: **if** $T_{\text{NEXT}} \leq T_{\text{END}}$ **then**
 - 9: Call Algorithm 1 to insert $(T_{\text{NEXT}}, \text{typ}, \emptyset)$
 - 10: **end if**
 - 11: **Output:** Updated event list $\mathcal{E}$
-

Nearest-neighbour search. The platform uses a proximity-based dispatch rule. When a rider arrives, the closest available driver is selected. When a driver becomes available, the closest waiting rider is selected.

Algorithm 3 Find Closest Available Driver

- 1: **Input:** Pickup location O_j , available-driver set A
- 2: **Initialize:** Closest driver $i^* \leftarrow \emptyset$
- 3: **if** $A = \emptyset$ **then**
- 4: **return** $\emptyset$
- 5: **end if**
- 6: Select

$$i^* \leftarrow \arg \min_{i \in A} d(L_i, O_j)$$

- 7: **Output:** Closest available driver i^*
-

Algorithm 4 Find Closest Waiting Rider

- 1: **Input:** Driver i , waiting-rider set W
- 2: **Initialize:** Closest rider $j^* \leftarrow \emptyset$
- 3: **if** $W = \emptyset$ **then**
- 4: **return** $\emptyset$
- 5: **end if**
- 6: Select

$$j^* \leftarrow \arg \min_{j \in W} d(L_i, O_j)$$

- 7: **Output:** Closest waiting rider j^*
-

Trip construction. Once a driver and rider are matched, the trip characteristics must be computed. This includes the pickup distance, service distance, total distance, travel times, fare, operating cost, and net earnings.

Algorithm 5 Build Trip

- 1: **Input:** Rider j , driver i , driver location L_i , parameter set Θ
- 2: Compute pickup distance

$$m_{i,j}^{\text{pick}} \leftarrow d(L_i, O_j)$$

- 3: Compute service distance

$$m_j^{\text{serv}} \leftarrow d(O_j, D_j)$$

- 4: Compute total distance

$$m_{i,j}^{\text{tot}} \leftarrow m_{i,j}^{\text{pick}} + m_j^{\text{serv}}$$

- 5: Sample pickup time $\tau_{i,j}^{\text{pick}}$
- 6: Sample service time τ_j^{serv}

- 7: Compute total trip time

$$\tau_{i,j}^{\text{tot}} \leftarrow \tau_{i,j}^{\text{pick}} + \tau_j^{\text{serv}}$$

- 8: Compute fare

$$F_j \leftarrow f_{\text{base}} + c_{\text{mile}} m_j^{\text{serv}}$$

- 9: Compute operating cost

$$C_{i,j} \leftarrow c_{\text{petrol}} m_{i,j}^{\text{tot}}$$

- 10: Compute net earnings

$$N_{i,j} \leftarrow F_j - C_{i,j}$$

- 11: **Output:** Trip record $\tau_{i,j} = (m_{i,j}^{\text{pick}}, m_j^{\text{serv}}, m_{i,j}^{\text{tot}}, \tau_{i,j}^{\text{pick}}, \tau_j^{\text{serv}}, \tau_{i,j}^{\text{tot}}, F_j, C_{i,j}, N_{i,j})$
-

Assignment procedure. The assignment procedure converts a feasible driver-rider pair into an active trip. It updates the relevant states, records the match, constructs the trip, and schedules the pickup and dropoff events.

Algorithm 6 Assign Driver to Rider

```
1: Input: Driver  $i$ , rider  $j$ , current time  $T_{\text{NOW}}$ 
2: if driver  $i$  is offline or busy then
3:   return
4: end if
5: if rider  $j$  is not waiting or rider  $j$  is already matched then
6:   return
7: end if
8: Remove  $i$  from  $A$ 
9: Remove  $j$  from  $W$  if present
10: Mark driver  $i$  as busy
11: Mark rider  $j$  as matched
12: Record match  $(j, i, T_{\text{NOW}}, d(L_i, O_j))$ 
13: Construct trip record  $\tau_{i,j}$  using Algorithm 5
14: Append  $\tau_{i,j}$  to trip list  $\mathcal{T}$ 
15: Call Algorithm 1 to insert pickup event at  $T_{\text{NOW}} + \tau_{i,j}^{\text{pick}}$ 
16: Call Algorithm 1 to insert dropoff event at  $T_{\text{NOW}} + \tau_{i,j}^{\text{tot}}$ 
17: Output: Updated  $A$ ,  $W$ ,  $\mathcal{R}$ ,  $\mathcal{D}$ ,  $\mathcal{T}$ , and  $\mathcal{E}$ 
```

Driver rematching. Whenever a driver becomes available, the platform checks whether a waiting rider can be matched immediately.

Algorithm 7 Try to Match an Available Driver

```
1: Input: Driver  $i$ , current time  $T_{\text{NOW}}$ 
2: if  $i \notin A$  then
3:   return
4: end if
5: Find  $j^*$  using Algorithm 4
6: if  $j^* \neq \emptyset$  then
7:   Call Algorithm 6 for driver  $i$  and rider  $j^*$ 
8: end if
9: Output: Updated simulation state
```

Main Event-Driven Simulation

After defining the shared procedures, the main simulation algorithm can be presented more compactly. It repeatedly removes the earliest event from the priority queue, advances the simulation clock, and updates the system state according to the event type. Because the event loop is long, it is presented across two pages.

The main algorithm combines the demand side, the supply side, and the event logic of the simulation. Driver arrivals add supply to the platform, rider arrivals create demand, patience-expiry events model abandonment, and pickup and dropoff events describe the operational progression of active trips. If a driver remains online after completing a trip, the driver is returned to the available pool and may be matched again immediately.

Algorithm 8 Event-Driven Ride-Hailing Simulation

```
1: Input: Start time  $T_0$ , horizon  $H$ , rates  $\lambda_d, \lambda_r, \lambda_p$ 
2: Initialize:  $T_{\text{NOW}} \leftarrow T_0$ ,  $T_{\text{END}} \leftarrow T_0 + H$ 
3: Initialize  $A \leftarrow \emptyset$ ,  $W \leftarrow \emptyset$ 
4: Initialize driver states  $\mathcal{D}$ , rider states  $\mathcal{R}$ , match list  $\mathcal{M}$ , trip list  $\mathcal{T}$ 
5: Initialize counters  $\text{Comp} \leftarrow 0$ ,  $\text{Aban} \leftarrow 0$ 
6: Generate first driver arrival and first rider arrival
7: Call Algorithm 1 for both initial events
8: while  $\mathcal{E} \neq \emptyset$  do
9:   Remove earliest event ( $T_{\text{NEXT}}$ , type, info) from  $\mathcal{E}$ 
10:  if  $T_{\text{NEXT}} > T_{\text{END}}$  then
11:    break
12:  end if
13:   $T_{\text{NOW}} \leftarrow T_{\text{NEXT}}$ 
14:  if type = driver_arrival then
15:    Generate driver  $i$  with initial location  $L_i$ 
16:    Generate online duration  $H_i$ 
17:    Set planned offline time  $T_i^{\text{off}} \leftarrow T_{\text{NOW}} + H_i$ 
18:    Add driver  $i$  to  $\mathcal{D}$  and to  $A$ 
19:    Call Algorithm 1 to insert offline-attempt event at  $T_i^{\text{off}}$ 
20:    Call Algorithm 7 for driver  $i$ 
21:    Call Algorithm 2 for next driver arrival
22:  end if
23:  if type = rider_arrival then
24:    Generate rider  $j$  with pickup  $O_j$  and dropoff  $D_j$ 
25:    Generate patience time  $P_j$ 
26:    Set patience deadline  $T_j^{\text{pat}} \leftarrow T_{\text{NOW}} + P_j$ 
27:    Add rider  $j$  to  $\mathcal{R}$ 
28:    Find  $i^*$  using Algorithm 3
29:    if  $i^* = \emptyset$  then
30:      Add rider  $j$  to  $W$ 
31:      Call Algorithm 1 to insert patience-expiry event at  $T_j^{\text{pat}}$ 
32:    else
33:      Call Algorithm 6 for driver  $i^*$  and rider  $j$ 
34:    end if
35:    Call Algorithm 2 for next rider arrival
36:  end if
37:  if type = patience_expire then
38:    Let rider be  $j$ 
39:    if  $j \in W$  then
40:      Remove  $j$  from  $W$ 
41:      Mark rider  $j$  as abandoned
42:       $\text{Aban} \leftarrow \text{Aban} + 1$ 
43:    end if
44:  end if
```

Algorithm 9 Event-Driven Ride-Hailing Simulation (continued)

```
45:   if type = pickup_done then
46:     Let rider be  $j$ 
47:     if rider  $j$  is matched and pickup time is not yet recorded then
48:       Record pickup time of rider  $j$ 
49:       Mark rider  $j$  as picked up
50:     end if
51:   end if
52:   if type = dropoff_done then
53:     Let rider be  $j$ , driver be  $i$ , and trip index be  $k$ 
54:     Record dropoff time of rider  $j$ 
55:     Mark rider  $j$  as completed
56:     Update driver  $i$  location to  $D_j$ 
57:     Mark driver  $i$  as not busy
58:     Add net earnings from trip  $k$  to driver  $i$ 
59:      $\text{Comp} \leftarrow \text{Comp} + 1$ 
60:     if driver  $i$  is pending offline and  $T_{\text{NOW}} \geq T_i^{\text{off}}$  then
61:       Mark driver  $i$  offline
62:       Remove  $i$  from  $A$ 
63:     else
64:       Add  $i$  to  $A$ 
65:       Call Algorithm 7 for driver  $i$ 
66:     end if
67:   end if
68:   if type = driver_offline_attempt then
69:     Let driver be  $i$ 
70:     if driver  $i$  is busy then
71:       Mark driver  $i$  for exit after current trip
72:     else
73:       Mark driver  $i$  offline
74:       Remove  $i$  from  $A$ 
75:     end if
76:   end if
77: end while
78: Output: Driver states, rider states, trip list, match list, completed trips, and abandoned riders
```

Results

The purpose of the basic model was to provide a benchmark against which later calibrated and improved models could be compared. In line with the standard simulation framework, the outputs recorded from each run were the quantities directly linked to the project objectives: request volume, completion volume, abandonment volume, rider waiting times, and driver earnings.

In the basic model, the principal assumptions provided by BoxCar were used without empirical adjustment. In particular, driver and rider arrivals were generated from the originally assumed exponential distributions, initial driver locations and rider pickup/drop-off points were taken to be uniformly distributed over Squareshire, driver online durations were sampled from Uniform(5, 8), and realised trip times were generated using the original journey-time multiplier assumption.

The average results from repeated simulation runs of the basic model are summarised in the table below.

Metric	Empirical data	Basic model
Total drivers	4719	2984.50
Total requests	34421	29820.83
Completed journeys	34117	21890.81
Abandonments	304	7908.90
Abandonment rate (%)	0.883	26.520
Total driver earnings (£)	603581.12	441279.80
Average driver earnings (£)	127.90	147.86
Average rider waiting time (minutes)	14.20	24.85

Table 1: Comparison of empirical outcomes with the basic model

The baseline results showed clear mismatches with the observed system behaviour. In particular, the basic model did not capture the spatial concentration of drivers and riders, which led to unrealistic matching patterns and journey lengths. As a result, the model substantially underestimated completed journeys, overestimated abandonment, and produced rider waiting times that were much higher than those observed in practice. These discrepancies indicate that the uncalibrated model is useful primarily as an initial reference point, rather than as a reliable representation of the live BoxCar system.

2 Data Analysis

It is evident that the basic model, derived from meetings and discussions with BoxCar, is inadequate to accurately simulate the interactions and actions on their app. Therefore we carried out extensive data analysis on the provided datasets detailing driver and rider information in order to find and develop distributions that more accurately reflected both the datasets and what we would expect to see in reality.

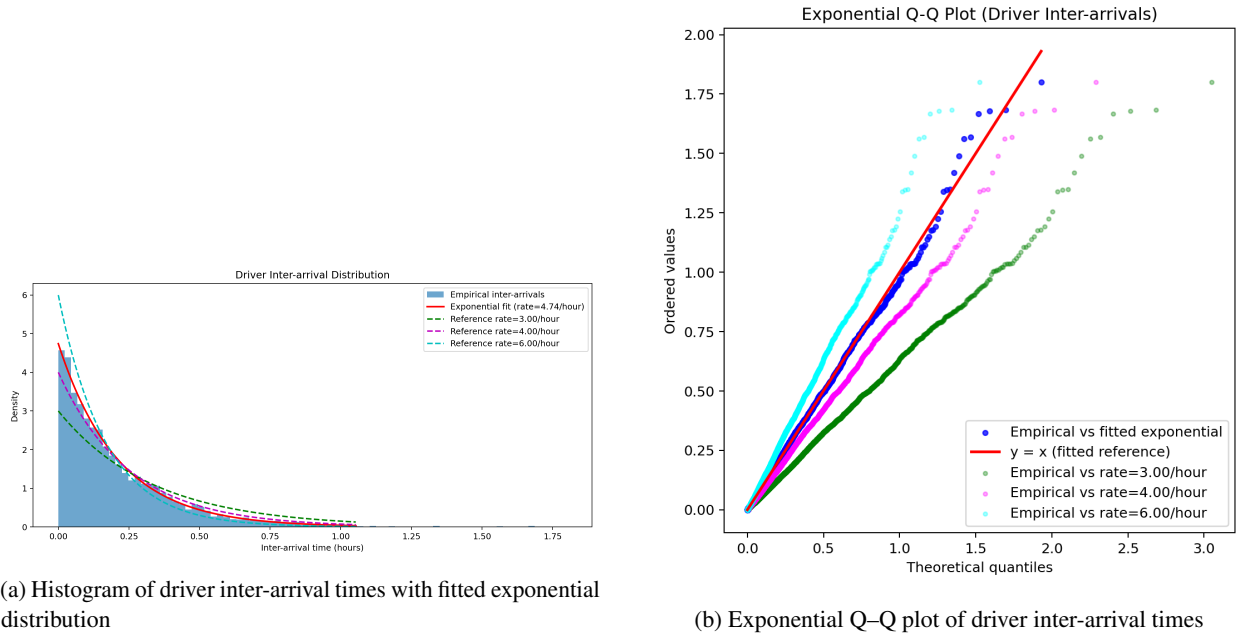
Driver Inter-arrival and Rider Inter-request Rates

The basic model placed the inter-arrival rate of drivers making themselves available in the app at exponential(3/hour). To test this we calculated the consecutive inter-arrival times from the provided data and from the subsequent 4718 data values we calculated the sample mean inter-arrival time of

$$\bar{x} = 0.2109 \text{ hours}$$

. This corresponds to a fitted rate of

$$\hat{\lambda} = \frac{1}{\bar{x}} = 4.7423 \text{ per hour}$$



(a) Histogram of driver inter-arrival times with fitted exponential distribution

(b) Exponential Q-Q plot of driver inter-arrival times

Figure 1: Empirical validation of the exponential distribution for driver inter-arrival times.

The Histogram of Exponential Q-Q plot in Figure 1 demonstrate that driver inter-arrival times do indeed follow and exponential distribution. They also demonstrate that using a rate of $\hat{\lambda} = 4.74$ fits the empirical data far better than the quoted rate of $\hat{\lambda} = 3$. This suggests that drivers are making themselves available on the app more frequently than believed by BoxCar.

There was similar under-estimation of the rider inter-request rate, which was placed at exponential(30/hour). From the rider data, we were able to obtain 34420 samples of inter-request times. We calculated the corresponding sample mean to be

$$\bar{x} = 0.0289 \text{ hours}$$

which corresponds to a fitted rate of

$$\hat{\lambda} = \frac{1}{\bar{x}} = 34.5961 \text{ per hour}$$

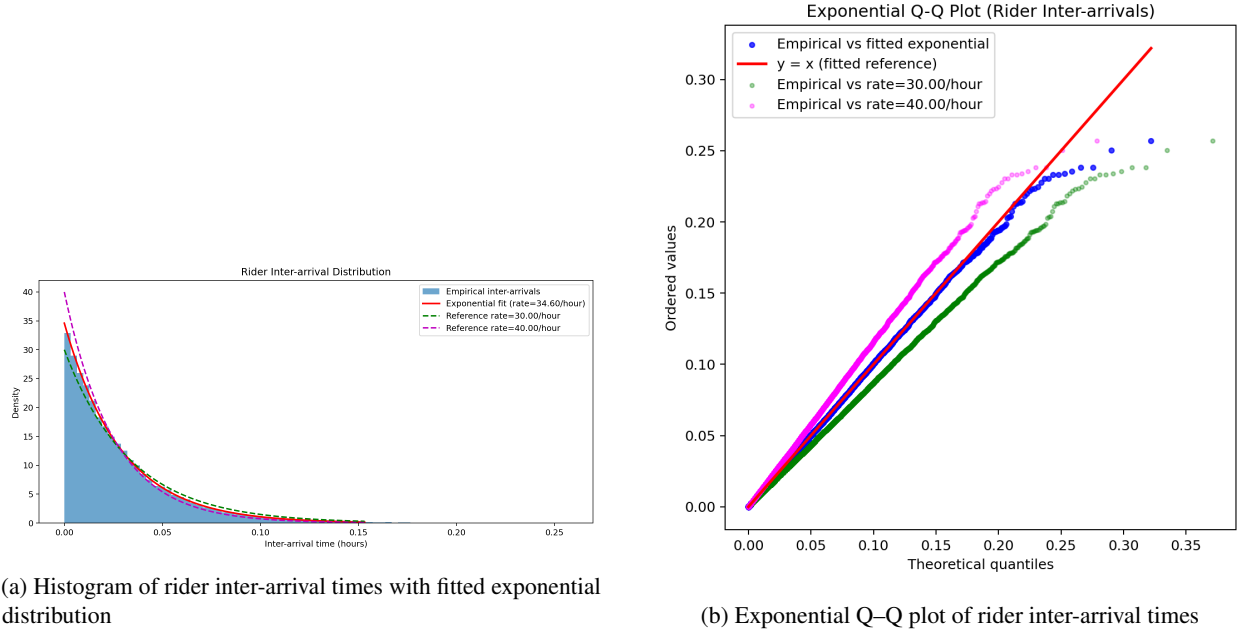


Figure 2: Empirical validation of the exponential distribution for rider inter-arrival times.

Figure 2 displays another Histogram and the Exponential Q-Q plot. These again demonstrate, firstly that it is suitable to model these times with an exponential distribution, and secondly that a rate of $\hat{\lambda} = 34.6$ fits the empirical data more accurately than the quoted rate of $\hat{\lambda} = 30$.

Driver and Rider Origins and Destinations

Initial discussions with BoxCar suggested that driver initial locations, as well as rider pickup and drop-off locations, were assumed to be uniformly distributed across Squareshire.

However, histograms of the observed x and y coordinates (Figures 3 and 4) indicated systematic deviations from uniformity. In particular, the spatial distributions exhibited clear skewness along both axes, suggesting that a uniform spatial model would not accurately reproduce the empirical patterns.

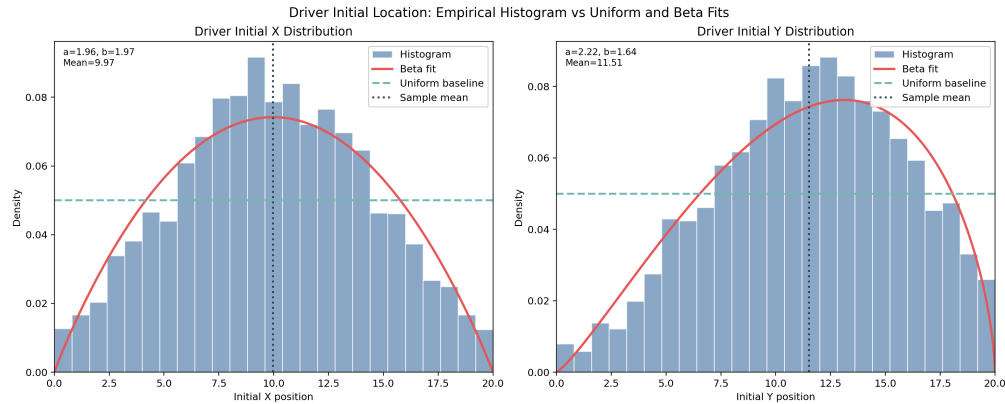


Figure 3: Fitted beta distributions for driver initial locations

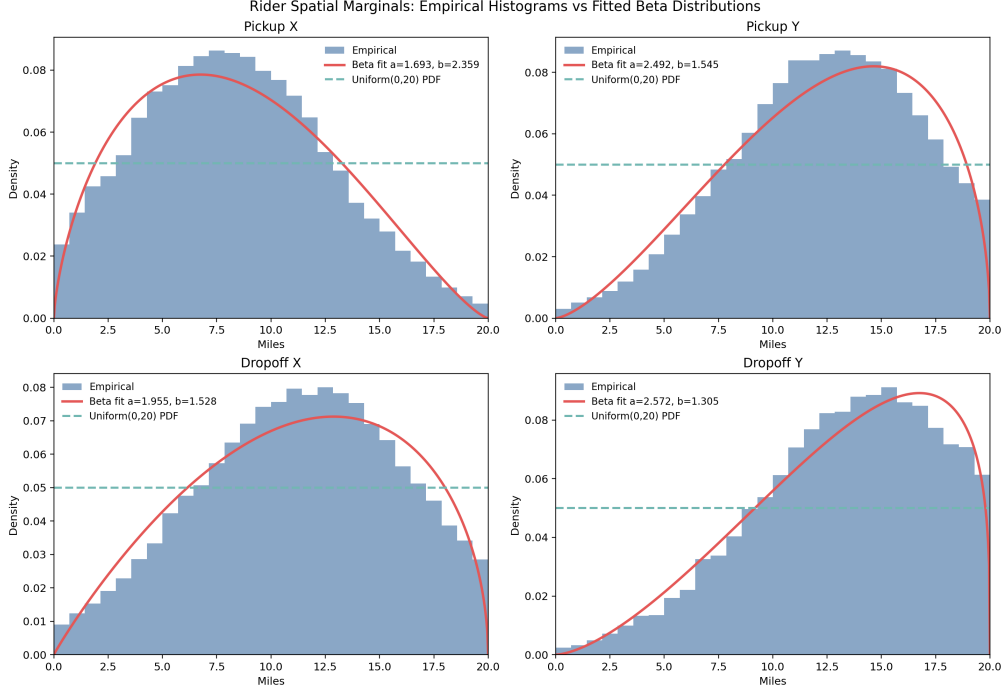


Figure 4: Fitted beta distributions for Rider pickup and drop-off locations

To capture these spatial effects, we modelled the normalised coordinates using Beta distributions. Let Z denote a normalised coordinate; we assume

$$Z \sim \text{Beta}(a, b).$$

Maximum likelihood estimation was used to estimate the parameters (a, b) for each coordinate distribution. The log-likelihood for a sample $\{z_i\}_{i=1}^n$ is given by

$$\ell(a, b) = (a - 1) \sum_{i=1}^n \log z_i + (b - 1) \sum_{i=1}^n \log(1 - z_i) - n \log B(a, b).$$

The estimated parameters are reported in Tables 2 and 3.

Drivers:

Coordinate	a	b
Initial x	2.74	2.75
Initial y	3.11	2.29

Table 2: Estimated Beta distribution parameters for driver initial locations.

Riders:

Coordinate	a	b
Pickup x	1.6927	2.3589
Pickup y	2.4916	1.5449
Dropoff x	1.9545	1.5276
Dropoff y	2.5716	1.3050

Table 3: Estimated Beta distribution parameters for rider pickup and drop-off locations.

To evaluate the adequacy of the fitted models, Kolmogorov–Smirnov (KS) statistics were computed between the empirical cumulative distribution functions and the corresponding fitted Beta distributions.

The results of these tests are detailed Tables 4 and 5

Drivers:

Variable	D
Initial x	0.027
Initial y	0.025

Table 4: Kolmogorov–Smirnov statistics for the driver fitted Beta distributions.

Riders:

Variable	D
Pickup x	0.023953
Pickup y	0.027133
Dropoff x	0.025000
Dropoff y	0.022706

Table 5: Kolmogorov–Smirnov statistics for the rider fitted Beta distributions.

The KS statistics are small across all coordinates, indicating that the Beta distributions provide a good fit to the observed spatial data.

This improvement is further illustrated in the heatmaps shown in Figures 5 and 6, where the modified simulation reproduces the spatial concentration patterns observed in the empirical data. In contrast, the original uniform assumption generates spatial distributions that are noticeably less consistent with the observed rider and driver location patterns.

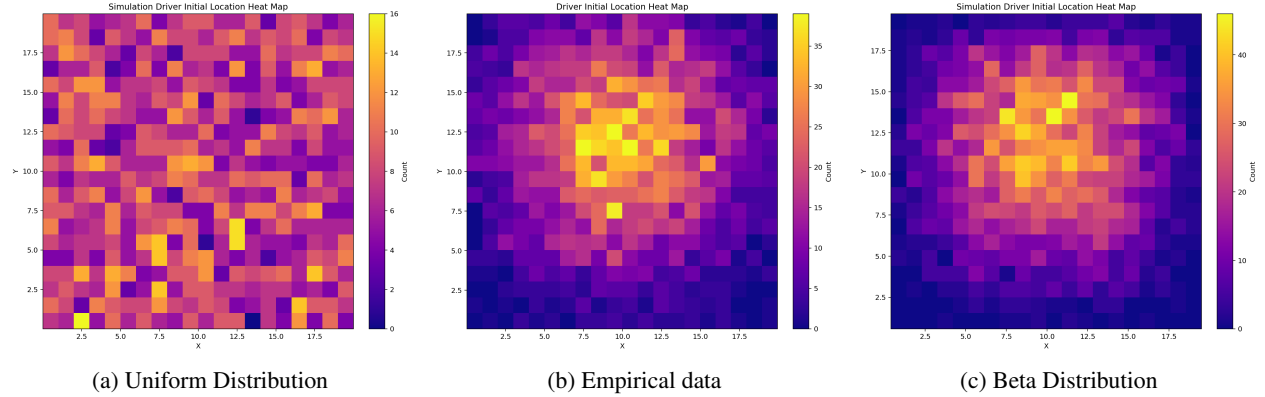


Figure 5: Comparison of driver initial location distributions from the empirical dataset with uniform and beta distributions.

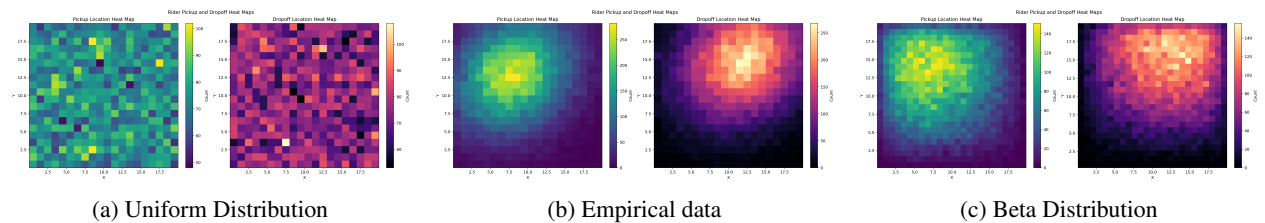


Figure 6: Comparison of rider pickup and drop-off location spatial distributions from the empirical dataset with uniform and beta distributions.

Driver Online Duration

The initial system description assumed that driver online durations were uniformly distributed between 5 and 8 hours. To evaluate this assumption, driver online times were calculated by subtracting the driver availability time from the recorded offline time.

This calculation was performed for both drivers who went offline immediately after completing a trip and those who were scheduled to go offline while still serving a rider. The inclusion of both categories increases the sample size, and the small extension caused by drivers completing their final trip before going offline was deemed negligible relative to the overall duration.

The histogram of the resulting online durations, shown in Figure 7, indicates that very few drivers remain online for less than six hours. Summary statistics further support this observation: the average online time was approximately 7.25 hours, with the 10th percentile at 6.36 hours and the 90th percentile at 8.09 hours.

Based on this empirical evidence, we propose replacing the original Uniform(5,8) assumption with a Uniform(6,8) distribution. As illustrated in Figure 7, this modification provides a noticeably better visual fit to the observed data.

To formally assess these candidate distributions, a chi-squared goodness-of-fit test was conducted using 12 equal-width bins, resulting in an expected count of approximately 390 observations per bin. The resulting test statistics are reported in Table 6. The very large statistic for Uniform(5,8) clearly indicates that this distribution is inconsistent with the observed data. In contrast, the substantially smaller statistic for Uniform(6,8) is consistent with sampling variability and supports its use in the simulation model.

Distribution	χ^2
Uniform(5,8)	2362.060
Uniform(6,8)	7.345

Table 6: Chi-squared goodness-of-fit statistics for candidate driver availability distributions.

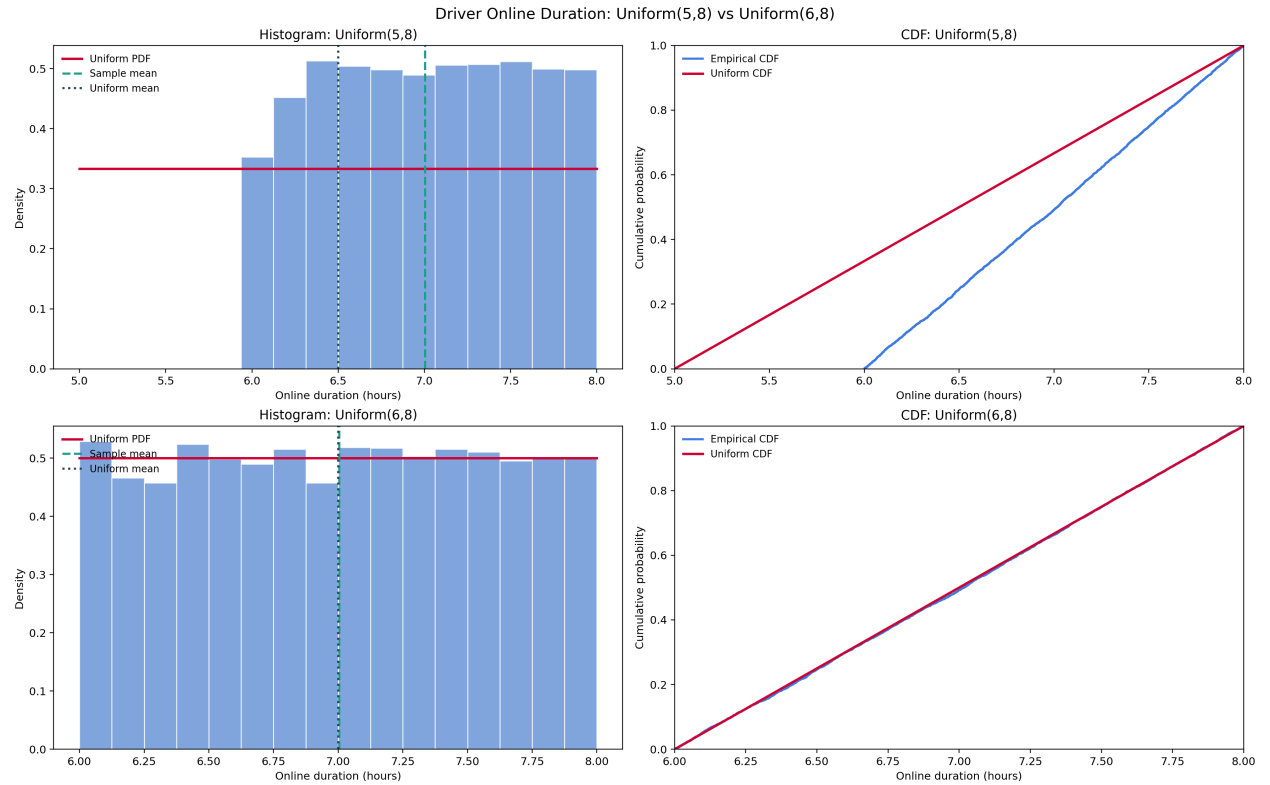


Figure 7: Comparison of Uniform(5,8) and Uniform(6,8) distributions of online time.

Journey Length

Average system speed was estimated by computing Euclidean service distances between rider pickup and drop-off locations. For each completed trip, the service time was calculated as the difference between the recorded drop-off time and pickup time. Using these values, the distance-weighted mean system speed was computed as total distance travelled divided by total service time. This resulted in an estimated average speed of 19.9751 mph. A one-sample test of the mean speed against the assumed value of 20 mph indicated that the observed difference is not statistically significant, supporting the continued use of 20 mph as the nominal system speed.

Under this assumption, the original model specified that the realised journey time was uniformly distributed between 0.8 and 1.2 times the expected travel time at this speed. However, analysis of the observed trip durations showed that this interval captured only approximately 66% of journeys. Expanding the multiplier range to Uniform(0.7, 1.3) covered the full range of observed journey times. This comparison is illustrated in Figure 8.

To formally compare the candidate distributions, a chi-squared goodness-of-fit test was performed using 12 equal-width bins. The resulting statistics, shown in Table 7, indicate that Uniform(0.7, 1.3) provides a substantially better fit to the observed journey time multipliers than the originally assumed Uniform(0.8, 1.2).

Distribution	χ^2
Uniform(0.7, 1.3)	6.386
Uniform(0.8, 1.2)	18.562

Table 7: Chi-squared goodness-of-fit statistics for candidate journey time multiplier distributions.

Average speed is approximately 20mph from testing Actual trip time is better uniformly distributed between U(0.7,1.3) compared to U(0.8,1.2)

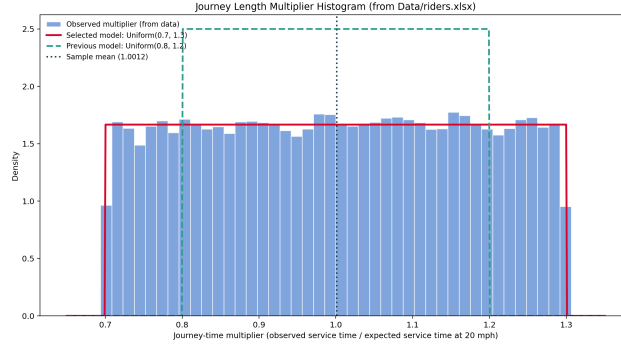


Figure 8: Comparison of Uniform(0.8,1.2) and Uniform(0.7,1.3) distributions of expected journey times.

Patience Time

The provided dataset does not contain sufficient information to directly estimate the distribution of rider patience times. In particular, the data only records whether a request was ultimately abandoned, rather than the precise time at which a rider decided to cancel.

To avoid unrealistic behaviour in the simulation, we therefore introduced a minimum patience threshold of six minutes before a rider may cancel a request. Extremely short patience times (e.g., cancellations within one or two minutes) are unlikely to represent genuine rider behaviour and are more plausibly explained by accidental requests, app interaction delays, or data recording artefacts.

Introducing this lower bound prevents unrealistically rapid cancellations and produces more behaviourally plausible simulation dynamics while preserving the overall abandonment behaviour observed in the dataset.

3 Updated model

The basic model provided a useful starting point for understanding the BoxCar platform, but the data analysis in Section 2 showed that several of the original assumptions were inconsistent with observed system behaviour. In particular, the baseline model understated both driver and rider arrival intensity, imposed unrealistic uniform spatial distributions on riders and drivers, and used distributional assumptions for online duration and realised journey times that did not match the empirical evidence. As a result, although the basic model captured the broad event logic of the platform, it did not reproduce the operational outcomes of the real system with sufficient accuracy.

To address this, the simulation was recalibrated using the empirical parameter estimates obtained from the driver and rider datasets. The updated model retains the same discrete-event structure, event list, object definitions, and state-transition logic as the basic model. This was intentional: the purpose of the calibration stage was not to redesign the system, but to improve the realism of the input processes so that the simulation outputs more closely match the live platform. Keeping the event logic fixed while updating only the input distributions also makes the calibration exercise easier to interpret, since any improvement in the model outputs can be attributed directly to the revised assumptions rather than to changes in system rules.

The following modifications were introduced in the updated model:

- Driver inter-arrival times were changed from Exponential(3/hour) to Exponential(4.7423/hour).
- Rider inter-request times were changed from Exponential(30/hour) to Exponential(34.5961/hour).
- Driver initial locations were sampled from fitted Beta distributions rather than uniformly over the region.
- Rider pickup and drop-off coordinates were sampled from fitted Beta distributions rather than uniformly over the region.
- Driver online durations were changed from Uniform(5,8) to Uniform(6,8).
- Journey-time multipliers were changed from Uniform(0.8,1.2) to Uniform(0.7,1.3).
- A minimum rider patience threshold of six minutes was imposed to avoid unrealistically short cancellations.

These changes were implemented modularly within the simulation engine. In practice, this meant replacing the random-generation functions associated with arrivals, locations, online durations, and trip-time variation, while leaving the event handlers unchanged. Drivers still enter the system, remain online for a random duration, accept jobs when available, complete pickups and drop-offs, and then either return to the available pool or leave the platform if their online period has expired. Riders still arrive stochastically, enter the waiting pool if unmatched, and leave either upon successful pickup or when their patience deadline is reached.

The updated model also incorporates a simulation warm-up period to remove the bias caused by the artificial empty starting state. Each run begins at time $t=0$ with no drivers online and no riders waiting, even though the real BoxCar platform typically operates with both already present in the system. If performance were measured from the outset, rider requests would arrive before the driver pool had built up, producing unrealistically high early abandonment and distorting the results away from the platform's steady-state behaviour. To reduce this initialisation bias, the first five hours of simulated time are excluded from the performance calculations, and all reported metrics are computed only from observations collected after this warm-up period. This allows the system to populate naturally with drivers, requests, and active journeys before measurement begins.

The purpose of developing the updated model was to reproduce the real system as faithfully as possible under the *current* operational rules. This gave us a calibrated benchmark which could be used to conduct policy experiments and, in turn, provide credible managerial recommendations.

Results of the updated model

Table 8 compares the observed empirical outcomes with the outputs of both the basic and updated models. This table provides a direct assessment of whether the calibration stage succeeded in producing a simulation that better reflects the behaviour of the real BoxCar system.

Metric	Empirical data	Basic model	Updated model
Total drivers	4719	2984.50	4759.61
Total requests	34421	29820.83	34456.75
Completed journeys	34117	21890.81	34091.11
Abandonments	304	7908.90	342.41
Abandonment rate (%)	0.883	26.520	0.994
Total driver earnings (£)	603581.12	441279.80	583960.30
Average driver earnings (£)	127.90	147.86	122.69
Average rider waiting time (minutes)	14.20	24.85	14.78

Table 8: Comparison of empirical outcomes, the basic model, and the updated calibrated model.

The calibrated model performs substantially better than the basic model across all of the main operational indicators. Most importantly, the updated model reproduces the overall scale of system activity much more accurately. The simulated number of drivers rises from 2984.50 in the basic model to 4759.61 in the updated model, which is very close to the empirical total of 4719. Similarly, the number of rider requests increases from 29,820.83 to 34,456.75, closely matching the observed total of 34,421. This improvement reflects the revised estimates for the driver and rider arrival processes, both of which were shown in Section 2 to be more intense than BoxCar originally believed.

The strongest evidence of improvement is seen in journey completion and abandonment outcomes. In the basic model, only 21,890.81 journeys are completed on average, compared with 34,117 in the real system. This implies that the baseline model was severely understating platform throughput. After calibration, completed journeys increase to 34,091.11, which is extremely close to the empirical total. Likewise, abandonments fall dramatically from 7908.90 in the basic model to 342.41 in the updated model, compared with 304 in the observed data. In proportional terms, the abandonment rate drops from an unrealistic 26.52% in the basic model to 0.994% in the updated model, closely matching the empirical rate of 0.883%.

This improvement is operationally very important. The very high abandonment rate in the basic model suggested a system suffering from chronic driver shortage and inefficient dispatch. However, the empirical data does not support such a conclusion. Instead, the calibrated model shows that once realistic arrival rates, spatial patterns, and journey dynamics are used, the current BoxCar platform appears much more capable of serving demand effectively. The implication is that the poor performance of the basic model was driven primarily by incorrect assumptions rather than by inherently flawed event logic.

Average rider waiting time shows a similar pattern. In the empirical dataset, average waiting time is 14.20 minutes, while the basic model predicts 24.85 minutes. This overstatement is substantial and would lead to misleading managerial conclusions if taken at face value. In contrast, the updated model reduces the average waiting time to 14.78 minutes, which is close to the real value. This improvement is particularly significant because waiting time is one of the key measures of rider experience, and it is closely linked to abandonment risk. A model that fails to reproduce waiting times credibly cannot be relied upon for testing service-level changes.

The earnings results also become more realistic after calibration. Total driver earnings rise from £441,279.80 in the basic model to £583,960.30 in the updated model, much closer to the observed total of £603,581.12. Average driver earnings in the updated model (£122.69) are also much closer to the empirical value (£127.90) than the basic-model figure (£147.86). This matters because driver earnings are a core measure of platform sustainability: a model that overstates earnings may incorrectly suggest that drivers are better compensated than they actually are, while a model that understates them may exaggerate the need for fare redesign. The calibrated model provides a more balanced and credible estimate of actual driver performance.

Taken together, these results indicate that the updated model is a valid and much more reliable representation of the real BoxCar system than the original baseline model. The overall demand level, supply level, abandonment rate, waiting time, and driver earnings are all reproduced with much smaller error. In simulation terms, the calibration stage can therefore be regarded as successful: the updated model preserves the original operational logic of the platform while reproducing the key empirical performance measures far more accurately.

4 Operational improvement model

The purpose of this stage was to evaluate whether alternative dispatch and pricing policies could improve system performance while maintaining the realistic behavioural assumptions established during the calibration stage.

Because the calibrated model already contained the required event structure, state variables, and journey dynamics, these policy experiments could be implemented without redesigning the entire simulation. Instead, only the matching logic and the fare calculation needed to be modified. This illustrates one of the key advantages of modular discrete-event simulation models: once a realistic baseline system has been constructed, alternative operating policies can be tested within the same framework without rebuilding the model.

Three modifications were introduced.

1. The driver–rider matching rule was modified so that rider waiting time is considered alongside pickup distance when selecting assignments.
2. A reservation mechanism was introduced allowing riders to be matched with drivers who are currently busy but expected to complete their trips nearby within a short time window.
3. Additional surcharges were introduced for journeys involving excessively long pickup distances or unusually long service trips.

These modifications were designed to address two main inefficiencies observed in the calibrated model: avoidable rider abandonment and imperfect driver compensation on inefficient journeys.

Waiting-time aware matching

In the calibrated model, riders were matched to the nearest available driver. Although this rule is simple and geographically efficient, it can disadvantage riders who have already been waiting for some time if newer requests continue to appear closer to newly available drivers.

To reduce this effect, the matching rule was modified to combine proximity with waiting-time priority. Let $R(t)$ denote the set of riders waiting at time t , and let $A(t)$ denote the set of available drivers. For a given driver $i \in A(t)$ with current location L_i , define the set of nearby waiting riders as

$$N_i(t) = \{j \in R(t) : d(L_i, O_j) \leq r\},$$

where r is a predefined proximity radius.

If this nearby set is non-empty, the driver is assigned to the rider in that set who has waited the longest, that is,

$$j^* = \arg \min_{j \in N_i(t)} t_{r,j},$$

where $t_{r,j}$ is the request time of rider j . Since earlier request times correspond to longer waits, this rule gives priority to riders who have already spent more time in the queue. If no riders lie within the nearby radius, the system falls back to the original nearest-rider rule:

$$j^* = \arg \min_{j \in R(t)} d(L_i, O_j).$$

This hybrid rule preserves local geographic efficiency while reducing the likelihood that riders with relatively long waits are repeatedly overlooked.

Driver reservation mechanism

The second improvement introduces a limited reservation mechanism. In the original and calibrated models, a rider could only be matched with a driver who was idle at the moment the request arrived. However, this can be inefficient when a driver is about to complete a nearby trip and would shortly become the most suitable match.

Let $B(t)$ denote the set of drivers currently serving passengers at time t . For each busy driver $i \in B(t)$, let τ_i denote the remaining time until trip completion and let D_i denote the expected drop-off location.

A busy driver is treated as eligible for reservation if

$$\tau_i \leq \tau_{\max}$$

and

$$d(D_i, O_j) \leq r_{\text{reserve}},$$

where τ_{max} is the maximum acceptable remaining service time and r_{reserve} is a spatial threshold around the rider's pickup point.

If no idle driver is available but such a busy driver exists, the rider may be reserved for that driver. Once the ongoing trip finishes, the reserved rider is assigned immediately before the driver is returned to the general available pool. This mechanism allows the platform to make use of near-future driver availability rather than forcing the rider to remain unmatched until a later system event.

Operationally, this change is intended to reduce avoidable abandonment during short-lived shortages of idle drivers, especially in cases where the shortage is local rather than system-wide.

Revised fare structure

The third improvement modifies the fare structure to better compensate drivers for inefficient assignments. In the original model, the passenger fare depended only on the service distance between pickup and drop-off. This means that drivers received no additional compensation for unusually long unpaid pickups, and only limited compensation when a completed trip moved them into an area where future demand was relatively weak.

Let d_{pickup} denote the distance between the driver's current location and the rider pickup point, and let d_{service} denote the distance between pickup and drop-off. The base fare is

$$F_{\text{base}} = \alpha + \beta d_{\text{service}},$$

where α is a fixed charge and β is the per-mile fare rate.

Two surcharges are then added. The first compensates for long pickup distances:

$$F_{\text{pickup}} = \begin{cases} \gamma & \text{if } d_{\text{pickup}} > d_p^*, \\ 0 & \text{otherwise.} \end{cases}$$

The second compensates for trips whose drop-off locations are far from the main pickup-demand hotspot H :

$$F_{\text{drop}} = \begin{cases} \delta & \text{if } d(D_j, H) > d_h^*, \\ 0 & \text{otherwise.} \end{cases}$$

The resulting total fare is therefore

$$F = \alpha + \beta d_{\text{service}} + F_{\text{pickup}} + F_{\text{drop}}.$$

This pricing adjustment is particularly relevant in Squareshire because demand is spatially concentrated rather than evenly spread across the region. As shown in the earlier spatial analysis, pickup and drop-off activity tends to cluster in certain areas. Consequently, excessively long journeys are not just costly because of extra travel distance; they may also move drivers away from the parts of the map where they are most likely to receive another request. Such trips therefore impose an opportunity cost on drivers by reducing access to future passengers. The surcharge structure is intended to compensate drivers for both the direct cost of inefficient travel and the indirect loss of future earning opportunities.

Operational interpretation

Taken together, these changes alter the behaviour of the platform in three important ways. First, riders who have already waited for longer are less likely to be displaced repeatedly by newly arriving nearby requests. Second, drivers who are about to complete nearby journeys can be utilised through reservation, reducing rider delay when no idle driver is immediately available. Third, drivers receive better compensation for assignments that are geographically or operationally inefficient.

The overall objective is not simply to increase completion numbers, but to improve the balance of the system. A better-performing platform should reduce rider abandonment, maintain or improve waiting times, and produce stronger and more stable driver earnings without undermining spatial matching efficiency.

Results of the operational improvement model

Table 9 compares the performance of the calibrated updated model with the operational improvement model. The objective of this comparison is to evaluate whether the revised dispatch and pricing rules lead to measurable improvements in both rider service quality and driver outcomes.

Metric	Updated model	Operational improvement model
Completed journeys	34091.11	34207.45
Abandonments	342.41	240.08
Abandonment rate (%)	0.994	0.696
Average rider waiting time (minutes)	14.78	14.72
Average pickup distance (miles)	4.82	4.78
Average driver hourly earnings (£/hour)	16.92	19.22
Std. dev. of driver hourly earnings (£/hour)	6.01	5.41

Table 9: Performance comparison between the calibrated updated model and the operational improvement model. Driver outcomes are measured using hourly earnings in order to control for variation in driver online duration.

The operational improvement model produces clear improvements in overall system performance. Most importantly, rider abandonments fall substantially. The number of abandoned requests decreases from 342.41 in the updated model to 240.08 in the operational improvement model, with the abandonment rate falls from 0.994% to 0.696%. This suggests that the revised dispatch rule and reservation mechanism improve the platform’s ability to match riders with drivers, particularly during short periods in which no idle driver is immediately available.

Rider waiting time also improves slightly. The average waiting time falls from 14.78 minutes to 14.72 minutes. While the reduction is small, it indicates that the lower abandonment rate is not being achieved by increasing waiting times elsewhere in the system. Instead, the revised matching logic appears to allocate drivers more effectively across the platform.

Spatial efficiency is also maintained. The average pickup distance decreases slightly from 4.82 miles to 4.78 miles, indicating that drivers are not required to travel further before collecting passengers. This is an important finding, since it suggests that the improved outcomes are generated by better assignment decisions rather than by imposing additional travel on drivers.

Driver performance is evaluated using average hourly earnings rather than average total earnings per driver. This is a more appropriate metric because drivers remain online for different lengths of time, and total earnings may therefore give a misleading impression of compensation efficiency. Measured on an hourly basis, average driver earnings increase from £16.92 to £19.22 per hour under the operational improvement model.

In addition to the rise in average hourly earnings, the standard deviation of driver hourly earnings falls from £6.01 to £5.41 per hour. This is an important result because it suggests that earnings are distributed more evenly across drivers under the revised system. In other words, the operational improvement model appears not only to improve average driver compensation, but also to produce a fairer allocation of earnings across the driver population.

Overall, the operational improvement model delivers a more balanced outcome for the platform. Riders experience fewer abandoned requests and slightly shorter waiting times, while drivers benefit from higher and more stable hourly earnings. At the same time, the system preserves efficient spatial matching between riders and drivers. These results suggest that the proposed operational changes could improve the performance of the BoxCar platform without requiring any fundamental change to its underlying structure.